ASSIGNMENT 2

MEAN LAB

GURU NANAK DEV ENGINEERING COLLEGE

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF

BACHELOR OF TECHNOLOGY

(Information Technology)

SUBMITTED BY:

ANAND RAJ

University Roll No.: 2302957

SUBMITTED TO:

Prof. JAGDEEP SINGH

(Signature)

QUESTION 1

Aim: To construct a simple login page web application to authenticate users using the MERN stack.

Tech Stack: MongoDB, Express.js, React.js, Node.js.

GitHub Link: `https://github.com/iron-007/smartmess-managment`

Screenshots:

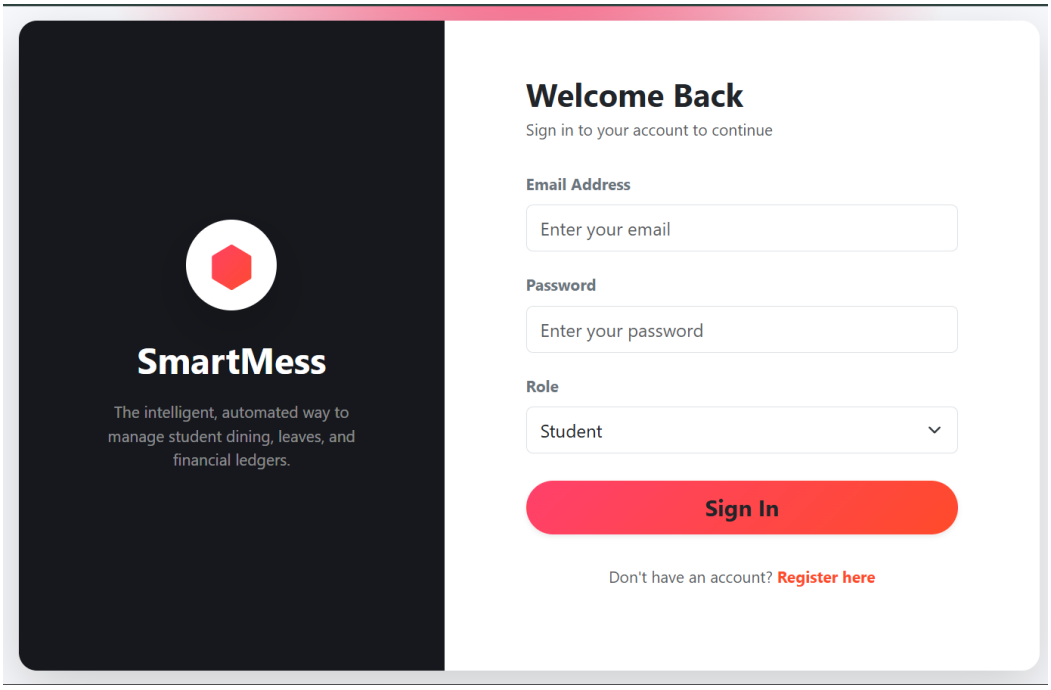

Figure 1: Login Page - Screen 1

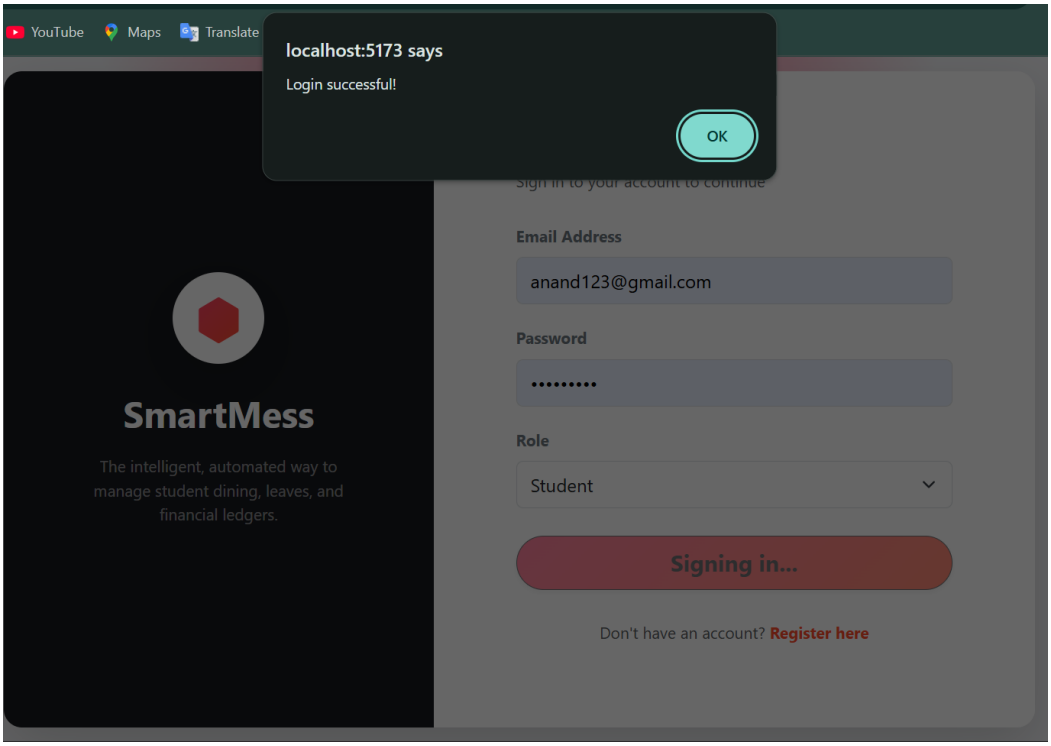

Figure 2: Login Page - Screen 2

QUESTION 2

Aim: Configure an Express application with a coding example.

File Structure:

```
EXPRESSJS AND EJS
 node_modules
 public
 views
 data.json
 .gitignore
 index.js
 package-lock.json
 package.json
```

GitHub Link: `https://github.com/iron-007/Expressjs-and-Ejs`

Code Implementation:

```
// ==========================================
// 1. SETUP AND IMPORTS
// ==========================================

// Import the Express library, which provides the tools needed to build our web
    server.
const express = require("express");

// Create an instance of the Express application. This 'app' object will be used
// to configure our server, set up routing, and handle incoming requests.
const app = express();

// Define the port number our server will listen to. Port 3000 is standard for
    local development.
const port = 3000;

```

```
// Import the built-in Node.js 'path' module. This is crucial for safely handling file paths
// across different operating systems (like Windows vs. Mac/Linux) without breaking the app.
const path = require("path");


// ==========================================
// 2. STATIC FILES CONFIGURATION
// ==========================================

// Tell Express to serve static files (like our custom CSS and JavaScript) from the 'public' folder.
// Using path.join(__dirname, ...) ensures the app always finds the correct folder,
// no matter where the server is launched from.
app.use(express.static(path.join(__dirname, '/public/css')));
app.use(express.static(path.join(__dirname, '/public/js')));

// Create specific routes for Bootstrap files.
// Instead of copying Bootstrap code into our public folder, we are directly linking
// the '/css' and '/js' browser paths to the Bootstrap files inside 'node_modules'.
app.use('/css', express.static(path.join(__dirname, 'node_modules/bootstrap/dist/css')));
app.use('/js', express.static(path.join(__dirname, 'node_modules/bootstrap/dist/js')));


// ==========================================
// 3. VIEW ENGINE (EJS) CONFIGURATION
// ==========================================

// Set EJS (Embedded JavaScript) as our templating engine.
// This allows us to write HTML pages that can display dynamic JavaScript data.
app.set("view engine", "ejs");
```

```

// Explicitly tell Express where to find our EJS template files.
// We are pointing it to the 'views' directory located in the current project folder.
app.set("views", path.join(__dirname, "/views"));


// ==========================================
// 4. ROUTE DEFINITIONS
// ==========================================

// --- Home Route ---
// When a user visits the root URL (http://localhost:3000/), this GET request triggers.
app.get("/", (req, res) => {
    // We use res.render() to send the 'home1.ejs' template to the browser.
    // Express automatically looks in the 'views' folder because of our configuration above.
    res.render("home1.ejs");
});

// --- Roll Dice Route ---
// A fun route that generates a random number and sends it to the webpage.
app.get("/rolldice", (req, res) => {
    // Generate a random integer between 1 and 6 to simulate a dice roll.
    let diceValue = Math.floor(Math.random() * 6) + 1;

    // Render the 'rolldice.ejs' file and pass the calculated diceValue variable to it
    // so it can be displayed dynamically on the screen.
    res.render("rolldice.ejs", { diceValue });
});

// --- Dynamic Instagram Profile Route ---
// This route uses a URL parameter (indicated by the colon ':') to create dynamic profile pages.
app.get("/ig/:username", (req, res) => {
```

```
    // Load our mock database of Instagram users from a local JSON file.
    const instaData = require("./data.json");

    // Extract the exact username the user typed into the URL (e.g., /ig/cats).
    const { username } = req.params;

    // Look up that specific username in our JSON data.
    const data = instaData[username];

    // If the user exists in our data file, render their profile page.
    if (data) {
        console.log(data); // Log the data to the terminal for debugging
        res.render("instagram.ejs", { data });
    }
    // If the username doesn't exist, render a custom error page instead of crashing.
    else {
        res.render("error.ejs");
    }
});


// ==========================================
// 5. SERVER INITIALIZATION & ERROR HANDLING
// ==========================================

// Tell the Express app to turn on and start listening for incoming requests on Port 3000.
// The callback function logs a message to the terminal so we know it started successfully.
app.listen(port, () => {
    console.log(`Listening on port ${port}`);
});

// --- 404 Catch-All Route ---
// This acts as a safety net. Express reads routes from top to bottom.
// If a user types a URL that doesn't match any of the routes above, they fall
```

```
    into this block,
// and we send back a 404 Not Found error to handle it gracefully.
app.use((req, res) => {
    res.status(404).send("Path does not exist.");
});
```